The Emptying — Phase 5 Technical Framework

Oct 26, 2026 · @William

The build-ready blueprint for Claude Code: architecture, a technical spec per system, data schemas, the build plan, and how systems are tweaked or removed. No code. The design stays locked; technical problems that touch the design are flagged for William to decide. This doc starts where the player does, at launch, and works inward.

Launch and front end

Two scenes, a main menu built over a slow 3D shot of the relic, a setup flow that ends in keep placement, and every interface written as code Claude Code can read. Status: in progress.

Scenes and flow

- Two scenes: **Main Menu** and **Game**. The main menu scene also loads settings and game data at launch.
- Flow: Launch → Main menu → New game setup → choose a keep site on the map → play. Side branches: tutorial, load, unlocks, settings, mods.

Main menu

- Items: Continue (last save), New Game, Load, Tutorial, Unlocks (races earned), Settings, Mods, Credits, Quit.
- **Look:** a slow 3D scene behind the menu. The relic sits in a dim keep, faintly pulsing, with distant ambient sound and music and the title above. Muted, dark palette: uneasy rather than horror. Built from Synty pieces.
- Mods need their own design discussion, since this is William's first modded game. Decided so far: the base game is built as a mod (a core data folder loaded like any other), which also makes official updates and expansions easy.

New game setup

Step	Choices
Kingdom	Name and banner (symbols and colors)
Race	Any unlocked race, showing strengths, weaknesses and starting resources
Faith	From the faiths allowed at start, or none
Difficulty	Easy, normal or hard preset, a customize panel, and ironman on or off
World	A world seed: random by default, or typed in to share or replay a world

The map then opens for keep placement, with the location popup.

Custom difficulty

- Sliders and toggles: event frequency, raid strength, starting gold, immigration rate, need drain speed, escalation speed, disease severity, how easily people defy laws, and whether downed people can die.
- **Slider registry:** every tunable value added or changed during development gets an entry in a running list, so the customize panel always covers the whole game.

Settings

Tab	Contents
Graphics	Resolution, window mode, quality preset, frame cap, vsync, render scale, shadows
Audio	Master, music, effects, ambient and interface volumes
Gameplay	Time scale (12 minutes per day by default), autosave interval and count, pause on alerts, tutorial hints, camera speed, edge scrolling
Controls	Rebind every key, mouse sensitivity

Tab	Contents
Accessibility	Text size, colorblind palette, reduced motion
Language	English for now, set up for translation

Loading screens and the herald

- Loading screens show tips and short lore fragments about relics and the being, written so a careful player pieces together the wider story across games.
- **The herald:** a system that tracks notable in-game events and calls them out, such as "Yesterday the queen had a group of cultists executed."

Interface technology

- **UI Toolkit** for menus and management screens: layout and style live in plain text files, so Claude Code writes and wires up the whole interface in code. The older UI system only for things floating in the 3D world, such as name tags.
- **Input System** for rebindable keys; settings saved as a small JSON file; the **Localization** package for future translation.

Terrain generation

The seed generates height, biomes and water on Unity Terrain tiles, then places hand-made points of interest by rule, 7 Days to Die-style. Status: first pass complete.

Map sizes and biomes

- Three sizes on the setup screen: small, medium (about 4 by 4 km, the default) and large. Larger maps take longer to generate and use more memory.
- Biomes: plains, forest, magic forest, mountains, swamp, ashen wastes, and coast or lakeland. Each has its own resources, threats, disasters, weather, immigrants and points of interest.

Generation steps

1. **Height:** layered noise for mountains, hills and valleys.
2. **Climate:** temperature and moisture maps decide each spot's biome.
3. **Water:** rivers trace downhill; lakes fill low basins.

4. **Regions:** named regions by biome, each with its own spawn tables.
5. **Points of interest:** each hand-made point of interest has a footprint, biome list, danger tier and spacing rules. The generator flattens the ground beneath it and blends the edges; tougher ones go farther from good starting land. A density slider controls how crowded the map is.
6. **Dressing:** Synty trees, rocks, grass and resource deposits placed by biome and slope.
7. **Start sites:** 5 temporary, empty sites on slightly flattened ground (water nearby, food and wood in reach). Once the player picks one, the other 4 are removed and their land returns to natural.
8. **Checks:** rules confirm a fair map, rerolling if one fails.

Locks and crossings

- No bridges are generated. People wade slowly through rivers; heavily used crossings get paved into bridges, through the paths system.
- **Relic shields** seal some regions and points of interest. Each shield is tied to a relic: find the relic that unlocks it, or find and destroy the relic powering it.
- Off-limits ground (cursed land, active cult land) can't take the keep.

In Unity

- Unity's built-in Terrain system, split into tiles for large maps.
- Pathfinding is built at runtime after generation and updates whenever the ground changes, such as when a house flattens the land beneath it.
- Heavy steps run on Jobs and Burst; target under a minute to generate a medium map.
- **Seed preview tool:** a developer tool that renders dozens of seeds as top-down images for quick tuning. Automated tests run thousands of seeds against the rule checks.

Architecture

Three layers, one simulation clock, systems that talk only through events, and every system a module that can be switched off. Status: first pass complete.

Layers

Layer	Contains	Knows about
Data	Content definitions (resources, buildings, laws, faiths), loaded from the core folder and mods	Nothing else
Simulation	Plain C# rules for every system: needs, laws, economy, combat	Data only, never Unity visuals
Presentation	Unity: models, animation, camera, UI, sound	Reads the simulation, sends player actions

The simulation never touches Unity directly, so off-screen people run as pure data, the rules are testable without opening the game, and saves stay clean.

Simulation clock

- A fixed tick, 10 per in-game minute by default.
- Each system chooses its own update rate: needs every tick, beliefs once per hour, the director a few times a day.
- Speed controls change ticks per real second; pause stops the clock.

Events

- Systems never call each other directly. They post events to a shared message board ("person died," "law changed," "building finished") and listen for the ones they care about.
- Removing a system just means its events stop; nothing else breaks. This reduces errors and hidden dependencies.

Modules

- Every system is a module with an on/off switch, a list of dependencies and its own settings.
- At startup the game checks dependencies: switching off religion also switches off cults and temples, with a clear message.
- This is the process for removing a system during development, and a player option where it makes sense.

Mods

- The base game is itself a mod: a core data folder loaded like any other.
- **Data mods and asset mods** from the start. Mods override content by ID: a mod that supplies a new keep model under the keep's ID is used instead of the original. Code mods much later, if ever.
- Steam Workshop support once the game is on Steam.
- A player-arranged load order where later mods win, with dependency and conflict warnings.
- Saves record the mods they used; missing mod content is dropped safely.
- **Custom behavior without code:** players can chain their own rules, such as a law leading to a punishment leading to a building, so a modder can sentence criminals to work on a pig farm they've added instead of executing them. This needs a visual, node-style rule system.

The herald

- Every event carries a notability score (executions, deaths of known people, new laws, raids and decrees score high; routines don't).
- At dawn the herald announces the top 1 to 3 events from the previous day; players set how chatty it is.

The project guide

- One guide file at the project root that Claude Code reads every session: the layers and their rules, folder structure, naming conventions, how to write tests, how to add a system, and links to the locked design docs.

The rule system

Every behavioral rule in the game is a chain of triggers, conditions and actions, authored in a node editor we use ourselves, so modders get exactly the tools we have. Status: first pass complete.

- Claude Code builds the framework and the editor; William authors the content with it. A strong editor is one of the things that can make the game stand out.
- Our own laws, punishments and events are built from the same blocks modders use. Execution is one action; a modder's pig farm sentence is another chain of the same blocks.

- The goal is complex but simple: powerful chains made from easy pieces.

Blocks

A starting set, grown over time:

Kind	Examples
Triggers	An event happens (crime known, death, building finished), a time of day, a value crosses a line (mood below 15), a law is discovered
Conditions	Compare a value (acceptance below 0), check a trait, race, faith or law setting, a random chance, "is known to the crown"
Actions	Change a value (mood, opinion, belief), assign a person to a building or job, apply a status (jailed, sapped), start an event, post a herald line, ask the player a question
Flow	And, or, not, wait, repeat, pick one at random

- **Decision popups are rules:** the first-theft popup is a trigger followed by an "ask the player" action, each option leading to its own chain. Modders can add their own popups the same way.
- **Tunable numbers:** any number in a rule can point to a named setting instead of a typed value ("theft fine amount"). Named settings automatically become sliders in the custom difficulty panel, keeping the slider registry current.

The editor

- A Unity editor window built on UI Toolkit, so the same editor can later ship inside the game for players.
- Rules are also stored as readable data files.
- **Test this rule:** runs a rule against a sample person or situation and shows each step: which condition passed, which action fired. It also flags duplicate rules and rules that contradict each other.

Running rules

- Rules compile into a fast internal form at load, rather than being interpreted from file each time they fire.
- Rules are checked for loops and impossible chains and capped in how often they fire, so a broken mod can't freeze the game. Bad rules show a clear error naming the mod.

Content importer

- A button in Unity reads the content spreadsheets (as CSV), checks every row (missing IDs, bad references, out-of-range values) and writes or updates the ScriptableObjects. Errors name the sheet, row and column.
- Mods can ship spreadsheets or JSON directly.

How systems are specified

Each system gets a spec, written right before its milestone so it reflects what we've learned while building.

Part	Contents
Purpose	One paragraph, linking to the design docs
Data it owns	Its fields and their ranges
Reads	What data it uses from other systems
Events	What it sends and listens for
Update rate	How often it ticks
Settings	Its sliders for the customize panel
Save data	What gets stored
Tests	What proves it works
Depends on	Modules it needs switched on

Order: core framework (clock, events, modules, data loading, the rule system, saves), then the gray-box prototype systems (people and needs, jobs and permissions, zones, town growth, storehouse, production chains), then the rest of the first playable version, then everything layered after.

Core framework

A deterministic tick, work spread by importance, end-of-tick events, per-module data keyed by ID, hard module boundaries, and debug tools from the first day. Status: first pass complete.

Clock and speed

- 10 ticks per in-game minute; at 12 real minutes per day that is 20 ticks per second at 1x, 40 at 2x and 60 at 3x. No faster speeds.
- At 3x with 2,000 people, a tick has about 16 milliseconds, so no system updates everyone every tick.

Updating by importance

- A scheduler gives each system an interval and staggers people across ticks, so load stays smooth.
- Each person has an importance score, recalculated cheaply every in-game hour, that sets how often they update. High: on screen, a lord or the ruler, a need near breaking, in a fight, committing or witnessing a crime, falling in love, near death. Low: content, healthy, working, nothing changing, like a happy farmer.

Determinism

- The simulation uses its own fixed tick and seeded random numbers, never frame rate or the system clock. The same save and the same inputs always play out the same way, so bugs can be reproduced.

Events and what the crown knows

- Events are small typed messages, queued during a tick and delivered at its end, so nothing chain-reacts mid-update.
- The rule system uses events as triggers; notable events feed the chronicle and the herald.
- **Not everything is known.** Events happen in the simulation whether or not anyone notices. The chronicle, the herald, laws and punishments see only what the crown knows, so coverage (patrols, guards, watchers) decides how much of your kingdom you can see. The developer console has a show-everything mode.

Visibility	Example	Known when
Public	A festival, a raid, a fire	Immediately
Witnessed	A brawl in the market	Someone saw it and reports it
Hidden	A murder behind a house	A witness, patrol, guard, detective or found evidence uncovers it; possibly days later, or never

- **Witnesses** report only if their opinion of the crown and their fear outweigh their loyalty to the person involved. A brother may hide his brother's crime.
- **Scores in the tools:** every event type carries scores set in the editor, such as theft at 5, which decide how readily it's reported and how notable it is. Player-made events get the same fields.

Modules and data

- A person is an ID; each module keeps its own table for that ID (Needs holds hunger and rest; Religion holds faith and devotion). Switching a module off removes its table without changing anything else.
- Each module has its own folder and assembly definition, and can use only the core and the modules it declares, so boundaries are enforced by the build.
- **Content definitions:** every content type (resource, building, trait, law, faith, event rule) is a ScriptableObject in the editor and a JSON file for mods; both load into the same internal form. IDs are readable text, such as resource.iron_ore or faith.old_halls, and never reused.
- **Saves:** each module saves and loads its own section. A save is a compressed binary file of labeled sections plus a version number, with a small readable header (kingdom name, date, population, mods used) for the load screen.

Developer tools

- From day one: an in-game console (spawn people, set needs, skip time, switch modules, fire events) and an inspector showing every module's data for whatever you click.

Failures

- In development, a module error stops the game loudly with the module's name.

- In release builds, a failing module is switched off and reported rather than crashing the colony.

Build plan

Seven milestones, each broken into self-contained prompts, with a check and a commit after every prompt. Status: first pass complete.

#	Milestone	Done when
0	Project setup	Unity project, folder structure, project guide, GitHub and test framework in place. Done November 7, 2026 (Unity 6000.6.8f1, URP; tests run from the command line)
1	Core framework	Clock, events, modules, data loading, saves, console and inspector work, with tests passing. Done November 7, 2026
2	Rule system and editor	Rules can be built in the editor, tested, and fire in the simulation
3	Gray-box town	On flat ground with cubes and capsules: people with needs, permissions, zones, a storehouse, 3 production chains, and people building homes and production on their own
4	Terrain generation	Seeds produce valid maps; points of interest and start sites placed; pathfinding works
5	First playable	Humans, discovered laws, influence tiers, crown buildings, one raid type with the alarm, main menu and setup
6+	Layers	Beliefs and campaigns, religion, businesses and corruption, fragments, off-map kingdoms, escalation, captives, races, then art

Milestone 3 tests whether the core idea is fun, so it comes before terrain.

Prompts

- Each prompt covers one coherent piece that can be tested on its own, roughly half a day to a day of Claude Code work.
- Format is whatever works best for that piece, from a few paragraphs to code blocks. Every prompt still states what it builds, what's out of scope, and how to verify it.
- **Between prompts:** William runs the tests, tries the checks and commits to GitHub before the next prompt, so problems never stack up.

Changes

- **Docs stay the source of truth.** When a prompt shows a spec was wrong or incomplete, the spec is updated in the same session; design-level problems go to William.
- **Mini review for every change or addition.** A new idea ("let's have people do this") gets scoped together first: what it touches, which modules and events it needs, whether it fits the locked design, and what it costs. Only then does it enter the docs and the build plan.

Milestone 1 specs: core framework

Eight core pieces, built in nine prompts, ending with a test that runs a day, saves, loads and proves the simulation is identical. Status: specs written November 7, 2026.

1. Seeded random

- **Purpose:** every random roll in the simulation comes from here, so the same seed always gives the same game.
- **Data:** a small, fast generator (PCG-style) with saved state. Each system gets its own named stream derived from the world seed plus the stream name ("needs", "events"), so adding rolls to one system never shifts another's results.
- **Provides:** next integer, next float, range, chance, weighted pick, shuffle.
- **Save data:** each stream's state.
- **Tests:** same seed gives the same sequence; different stream names give different sequences; save and restore continues the exact sequence; weighted picks match their weights over many rolls.

2. Clock and calendar

- **Purpose:** the simulation's only source of time.
- **Data:** the tick count. Everything else is derived: 10 ticks per minute, 60 minutes per hour, 24 hours per day, 15 days per season, 4 seasons per year.
- **Speed:** paused, 1x, 2x, 3x. Real seconds per in-game day is a setting (default 720, so 1x is 20 ticks per second).
- **The driver** lives in the presentation layer: it adds up real time and runs the right number of ticks each frame, with a cap so a slow frame never causes a runaway catch-up.
- **Events:** new hour, new day, new season, new year.
- **Tests:** tick-to-calendar conversion at boundaries; speed and pause change ticks per real second correctly; the catch-up cap holds.

3. Scheduler

- **Purpose:** spreads work across ticks so the load stays smooth.
- **Data:** each registered system has an interval and an offset. Per-entity work is split into buckets, a different bucket each tick.
- **Importance tiers:** high (every interval), normal (every 4th), low (every 16th), as starting values. A system asks for an entity's tier; the importance score itself arrives with the people modules.
- **Tests:** a system with interval 10 runs every 10 ticks; 1,000 entities across 10 buckets update about 100 per tick; tier multipliers apply.

4. Entities and module data

- **Purpose:** people, buildings and items are IDs; each module stores its own data per ID.
- **Data:** an entity ID (number plus a generation counter, so a reused slot can never be confused with an old one) and its kind (person, building, item and so on). A registry creates and destroys IDs.
- **Module data tables:** each module owns tables keyed by entity ID, with add, get, remove and iterate. Destroying an entity removes its rows from every table.
- **Events:** entity created, entity destroyed.
- **Tests:** stale IDs are rejected after reuse; destroy cleans every table; iteration is in a stable order (needed for determinism).

5. Event bus

- **Purpose:** the only way modules talk.
- **Data:** typed events, queued during a tick and delivered at the end of it, in the order posted. Events posted while delivering go into the next tick, so nothing chain-reacts.
- **Event definitions** (data): ID, visibility (public, witnessed, hidden), reporting score, notability score. The rules for who discovers hidden events come later; the fields exist now.
- **Tests:** delivery order; no same-tick cascades; unsubscribe works; definitions load with their scores.

6. Module system

- **Purpose:** every system is a switchable module.
- **Data:** each module declares an ID, version, dependencies, settings, and how it registers its systems, events, console commands and save section. A manifest per module says whether it's enabled.
- **Startup:** modules load in dependency order. A disabled module disables everything that depends on it, with a clear message. A missing dependency is an error naming both modules.
- **Failures:** in development, a module error stops the game and names the module. In release builds, the module is switched off and reported.
- **Tests:** dependency ordering; cascade disabling; circular dependencies are caught; a failing module in release mode is isolated.

7. Content and settings

- **Purpose:** load all content as data, from the core folder and mods.
- **Data:** every content type derives from one base with a permanent text ID. A content database stores everything by type and ID.
- **Loading:** ScriptableObjects from the core folder, JSON from mod folders. Each mod folder has a manifest (ID, version, dependencies). The load order is core first, then mods in the player's order; a later definition with the same ID replaces the earlier one.
- **Validation:** after loading, every reference is resolved and checked. Missing IDs, bad references and out-of-range values produce a clear list of errors naming the file.

- **Settings registry:** named settings (ID, default, minimum, maximum, category, description) that modules declare. This list is the slider registry and feeds the custom difficulty panel later.
- **Tests:** loading by ID; override by load order; validation catches each error type; settings stay within range.

8. Saves

- **Purpose:** save and load the whole simulation.
- **Format:** a small readable header (format version, game version, kingdom name, in-game date, population, world seed, mods used) followed by compressed sections, one per module, each with its own version number.
- **Rules:** modules save only their own data; a missing section (a module switched off) is skipped safely; a newer section version than the game understands is reported, not guessed.
- **Tests:** round trip (save, load, compare); a save with a module missing still loads; the header can be read without loading the rest.

9. Console, inspector and the game scene

- **Purpose:** see inside the simulation from the first day.
- **Game scene:** a bootstrap object creates the simulation, loads modules and content, and runs the clock driver.
- **Console:** opens with the backquote key, built with UI Toolkit. Modules register commands. Built-in commands: help, speed, pause, skip (hours or days), date, seed, module list, event list, spawn a test entity, inspect an entity, save, load.
- **Inspector:** shows every module's data for a selected entity. In milestone 1 it's opened from the console by ID; clicking objects comes with the gray-box town.
- **Tests:** commands parse and run; unknown commands show help.

Prompts

#	Builds
1.1	Seeded random, clock and calendar
1.2	Scheduler and importance tiers

#	Builds
1.3	Entities and module data tables
1.4	Event bus and event definitions
1.5	Module system
1.6	Content database, loaders, validation, settings registry
1.7	Saves
1.8	Game scene, clock driver, console and inspector
1.9	Milestone check: a small demo module, plus a test that runs one in-game day, saves, loads, runs another day, and proves the result matches an unbroken two-day run

Milestone 1 results

- Completed November 7, 2026: 125 EditMode and 12 PlayMode tests passing.
- A saved and reloaded game continues identically to an unbroken run (matching state hashes and event logs).
- Performance at 2,000 entities with the demo module: about 0.04 ms per tick on average (worst 4.5 ms, a single outlier), far inside the budget.
- Added along the way: a SimulationBuilder that runs startup the same way for new games and loads; module access to live settings; console fixes (theme, layout, focus, sticky scrolling); a module checklist in CLAUDE.md.

Carried forward:

- Per-tick allocations from table iteration, iterator wrappers and closures; measure properly in a later profiling pass.
- The console and HUD are built in C#; real game screens should use UXML and USS layout files.
- A few obsolete Unity API warnings to clean up.
- The temporary demo hunger module is deleted in milestone 3.

Milestone 2 specs: the rule system and its editor

Rules are Blueprint-style graphs stored as data, compiled at load into a fast runtime, edited in our own UI Toolkit node editor that can later ship in the game. Status: specs written November 8, 2026.

Decisions

- **Our own editor on UI Toolkit**, not Unity's Graph Toolkit, which is experimental and editor-only. One editor for us now and for players and modders later.
- **Blueprint-style rules:** execution wires decide what runs next; data wires pass values. Simple values can be typed into a node's fields or point to a named setting.
- **Blocks are C#,** registered by modules. Data mods combine existing blocks but can't create new kinds.
- **Paused rules are saved** and resume exactly on time after a load.
- **Questions to the player** queue up and pause the game by default (a setting); delegation to lords comes with the lords system.
- **Rule memory:** local variables while running, plus saved flags and counters on any entity or the kingdom.
- **Limits:** 1,000 fires per rule per tick, chains capped at 10 deep, loop warnings in the editor. In Release mode, a rule that breaks a limit is switched off and reported.
- The in-game version of the editor and system-specific blocks come in later milestones.

1. Rule data

- A rule is content of type "rule": Id, name, description, enabled, a list of nodes and a list of connections. Rules live in a source's Data folder like other content, so mods can add and override them.
- **Node:** a stable Id within the rule, its block type Id (such as "block.compare_number"), its position in the editor, and its field values (a typed literal, a reference to a named setting, or empty).
- **Ports:** execution ports (named, such as "then", "true", "false") and typed data ports.
- **Data types:** number, true or false, text, entity and content Id.
- **Connection:** from a node's output port to another node's input port. Execution outputs connect to execution inputs; data outputs to data inputs of the same type.

2. Blocks

- Each block type declares: Id, display name, category, kind (trigger, pure value, action or flow), its ports and fields, and how it runs.
- **Pure value blocks** (compare, math, read a setting) have no execution ports; they compute when something needs their output, as in Blueprints.
- Modules register block types during the declaration phase, in a block registry.

3. Compiling

- At load, each rule is checked and compiled into a fast internal form.
- **Errors:** unknown block types, type mismatches, a required input with neither a wire nor a value, more than one trigger, a missing trigger.
- **Warnings:** unreachable nodes; a rule that posts an event which triggers itself.
- Errors follow the content error rules: Development stops with the full list; Release skips the rule and reports it.

4. Running

- **Triggers:** an event (its parameters become data outputs), a time trigger (every N hours, at a given hour, each new day). More trigger types arrive with later systems.
- When a trigger fires, a rule instance is created and runs along its execution wires until it finishes, waits or asks the player.
- Event-triggered rules run when the event is delivered at the end of a tick. Events a rule posts are delivered the next tick, like every other event.
- Rules use their own named random stream, and instances run in creation order, so everything stays deterministic.
- Each event records how many rule steps led to it (chain depth), so the chain limit can be enforced.

5. Waiting, flags and counters

- **Wait:** pauses an instance for a number of in-game minutes, hours or days. Paused instances, with their local variables and the trigger's data, are saved and resume on the exact tick after a load.
- **Flags and counters:** named true-or-false flags and numeric counters on any entity or on the kingdom, read and written by blocks, saved with the game and shown by the inspector.

6. Asking the player

- An "ask" action block shows a question with options; each option is its own execution output.
- Questions queue and appear one at a time. A setting decides whether an open question pauses the game (default on).
- Pending questions are saved with the game.
- A hook exists for delegation, so a lord can answer by temperament once lords exist.

7. Core blocks for milestone 2

Category	Blocks
Triggers	On event, every N hours, at an hour of day, on new day
Values	Compare numbers, compare text, and, or, not, math (add, subtract, multiply, divide, min, max), random number, random chance, read setting, read event parameter
Flow	Branch (if), wait, sequence, stop
Memory	Get and set flag, get and change counter, local variables
Actions	Post event, write a log line, write a herald line, ask the player

8. The editor

- A node canvas built on UI Toolkit in its own assembly, so the same canvas can later run inside the game. In milestone 2 it opens as a Unity editor window.
- **Canvas:** pan, zoom, drag nodes, connect and disconnect wires (only matching types), select several, delete, undo and redo, copy and paste.
- **Nodes:** an add-node menu searchable by name and grouped by category; fields edited right on the node; a setting picker for fields that accept settings.
- **Files:** open, create and save rule files in a chosen source's Data folder.
- **Validation panel:** compile errors and warnings, each clickable to jump to its node.

9. Testing and tracing

- **Test this rule:** runs the rule in a sandbox simulation against a sample trigger (an event with parameters you fill in), then highlights each step on the canvas: which conditions passed and which actions fired.
- **Live trace:** console commands list rules, switch a live trace on or off for a rule, and show its recent runs.

Prompts

#	Builds
2.1	Rule data, content type, block registry, ports and types, the compiler's checks (no running yet)
2.2	Running rules: triggers, value evaluation, actions, branches, local variables, the first core blocks, limits
2.3	Wait with saved paused instances, flags and counters, determinism tests
2.4	Asking the player: the question queue, the popup, the pause setting, saving
2.5	The node canvas: pan, zoom, nodes, ports, wires, selection, dragging
2.6	The editor window: files, add-node menu, field editing, type-checked connections, undo and redo, copy and paste, validation panel
2.7	The test tool and live tracing
2.8	Milestone check: a reaction to the demo hunger module rebuilt as a rule, authored in the editor, plus a rule William builds himself